```
!pip install "gym==0.26.2"
!pip install "numpy<1.24"
```

```
Requirement already satisfied: gym==0.26.2 in /usr/local/lib/python3.12/dist-packages (0.26.2)
Requirement already satisfied: numpy>=1.18.0 in /usr/local/lib/python3.12/dist-packages (from gym=
Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: gym_notices>=0.0.4 in /usr/local/lib/python3.12/dist-packages (from
Collecting numpy<1.24
  Using cached numpy-1.23.5.tar.gz (10.7 MB)
  Installing build dependencies ... done
  error: subprocess-exited-with-error

  × Getting requirements to build wheel did not run successfully.
  │ exit code: 1
  ╰─> See above for output.

  note: This error originates from a subprocess, and is likely not a problem with pip.
  Getting requirements to build wheel ... error
error: subprocess-exited-with-error

× Getting requirements to build wheel did not run successfully.
│ exit code: 1
╰─> See above for output.

note: This error originates from a subprocess, and is likely not a problem with pip.
```

```
import numpy as np
np.bool8 = bool   # Fix for numpy error

import gym
```

```
env = gym.make("FrozenLake-v1", is_slippery=False)
```

```
state_size = env.observation_space.n
action_size = env.action_space.n

q_table = np.zeros((state_size, action_size))
```

```
alpha = 0.8      # Learning rate
gamma = 0.95     # Discount factor
epsilon = 1.0    # Exploration rate
episodes = 1000
```

```
for episode in range(episodes):
    state, _ = env.reset()
    done = False

    while not done:
        # Choose action
        if np.random.uniform(0,1) < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(q_table[state,:])

        # Take action (FIXED)
        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated

        # Update Q-table
        q_table[state, action] = q_table[state, action] + alpha * (
            reward + gamma * np.max(q_table[next_state,:]) - q_table[state, action]
        )
```

```
            state = next_state

        epsilon = epsilon * 0.995
```

```
print("Q-Table:\n", q_table)
```

```
Q-Table:
 [[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

```
episodes = 10000   # instead of 1000
```

```
import numpy as np
np.bool8 = bool # Fix for numpy error with gym
import gym
env = gym.make("FrozenLake-v1", is_slippery=False, new_step_api=True)
```

```
/usr/local/lib/python3.12/dist-packages/gym/core.py:317: DeprecationWarning: WARN: Initializing wr
  deprecation(
/usr/local/lib/python3.12/dist-packages/gym/wrappers/step_api_compatibility.py:39: DeprecationWarr
  deprecation(
```

```
alpha = 0.8
gamma = 0.95
epsilon = 1.0
```

```
import numpy as np

state_size = env.observation_space.n
action_size = env.action_space.n
q_table = np.zeros((state_size, action_size))

total_reward = 0

for episode in range(episodes):
    state = env.reset()
    if isinstance(state, tuple):
        state = state[0]

    done = False

    while not done:
        if np.random.uniform(0,1) < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(q_table[state,:])

        step_result = env.step(action)

        if len(step_result) == 5:
            next_state, reward, terminated, truncated, _ = step_result
            done = terminated or truncated
```

```
            done = terminated or truncated
        else:
            next_state, reward, done, _ = step_result

        q_table[state, action] = q_table[state, action] + alpha * (
            reward + gamma * np.max(q_table[next_state,:]) - q_table[state, action]
        )

        state = next_state
        total_reward += reward

    epsilon *= 0.995

print("Total Reward:", total_reward)
```

```
Total Reward: 9749.0
```

```
print("Q-Table:\n")
print(q_table)
```

```
Q-Table:

[[0.73509189 0.77378094 0.6983373  0.73509189]
 [0.73509189 0.         0.66325472 0.69721996]
 [0.6983373  0.         0.         0.62915982]
 [0.53073635 0.         0.         0.        ]
 [0.77378094 0.81450625 0.         0.73509189]
 [0.         0.         0.         0.        ]
 [0.         0.         0.         0.66342043]
 [0.         0.         0.         0.        ]
 [0.81450625 0.         0.857375   0.77378094]
 [0.81450625 0.9025     0.9025     0.        ]
 [0.857375   0.95       0.         0.63024941]
 [0.         0.         0.         0.        ]
 [0.         0.         0.         0.        ]
 [0.         0.722      0.95       0.6859    ]
 [0.9025     0.95       1.         0.9025    ]
 [0.         0.         0.         0.        ]]
```